

Close 歷史線型相似度驗證器

教練新手實戰教學手冊

Close Pattern Matching • Overlay Comparison • Historical Projection

完整使用教學手冊 | 適用新手理解、策略驗證、實戰判讀與參數調整



Close 歷史線型比對

模式	正常模式
參考開始	2026/06/28 07:00
參考結束	2026/06/30 06:00
正式終點	close[1] 已收盤 close[0] 排除
比對設定	48根 搜尋5000 間隔4/4 後續顯示100根
候選統計	已測1218 達157 門檻61.0%
更新/驗證	新 K 形成後更新 後續報酬不參與配對
狀態	已找到 3 組獨立歷史樣本
歷史 #1 80.9%	後續16根 -7.9%
2026/06/14 03:00 →	2026/06/16 02:00 距 340 根
歷史 #2 80.8%	後續16根 -4.9%
2026/01/01 07:00 →	2026/01/03 06:00 距 4272 根
歷史 #3 79.3%	後續16根 +1.29%
2026/03/14 19:00 →	2026/03/16 18:00 距 2532 根



只比較 Close 收盤價

聚焦 Close 價格型態，比對更純粹



正式終點固定 close[1]

close[1] 已收盤，close[0] 完全排除



重疊比對線

目前 Close 與歷史線型重疊比較



目前右側歷史後續投影

顯示歷史樣本後續 100 根走勢



前 1~3 組獨立歷史樣本

依相似度排序，供研判與參考



快速上手

從設定到讀圖，
1 小時快速掌握



指標架構

完整解析邏輯與
計算流程



實戰判讀

如何解讀投影結果
提升決策品質



手動調參

參數調整策略與
最佳化建議



手冊定位與閱讀方式

本手冊以「教練式新手實戰教學」為核心，目標不是只告訴你指標有哪些按鈕，而是完整拆解：這支 Pine Script 指標如何取得資料、如何建立參考區域、如何搜尋歷史候選、如何計算相似度、如何做樣本分離、為什麼要這樣判斷，以及每一個設定值改動後會對結果造成什麼影響。

本手冊採現代企業培訓手冊+極簡金融資訊圖表風設計，適用來快速上手、交接教學、課程講義與實戰複習。若你是第一次使用 Close 線型相似度工具，建議先依目錄順序閱讀；若你已經能操作指標，可直接跳到「參數調整」、「面板判讀」或「實戰情境」章節。

文件與著作權資訊

項目	內容
手冊名稱	Close 歷史線型相似度驗證器 練新手實戰教學手冊
程式作者／著作權人	Threads(@hongshihong19)
授權方式	Mozilla Public License 2.0
程式檔案	Close_歷史線型相似度驗證器_v1.2.2_Threads@hongshihong19.pine
文件語言／編碼	繁體中文 zh-TW／UTF-8
用途聲明	技術研究、資訊整理與交易流程訓練；不構成投資建議。

本手冊涵蓋的核心問題

- 指標到底在比較什麼？答案：只比較 Close 收盤價形成的線型路徑，不比較 K 棒色、實體或影線。
- 為什麼正式終點是 close[1] 而不是 close[0]？答案：因為 close[0] 未收盤，會造成盤中結果跳動與重繪觀感。
- 歷史相似區域是出來的？答案：先建立參考區域，再從歷史資料中依步長掃描候選，逐一計算相似度。
- 相似度是如何計算的？答案：由 Pearson 線型相關、正規化路徑距離、方向一致率三部分加權組成。
- 哪些地方可以手動調整？答案：比對長度、搜尋根數、候選間隔、隔離根數、最低相似度、驗證位移、歷史後續顯示根數等。

目錄

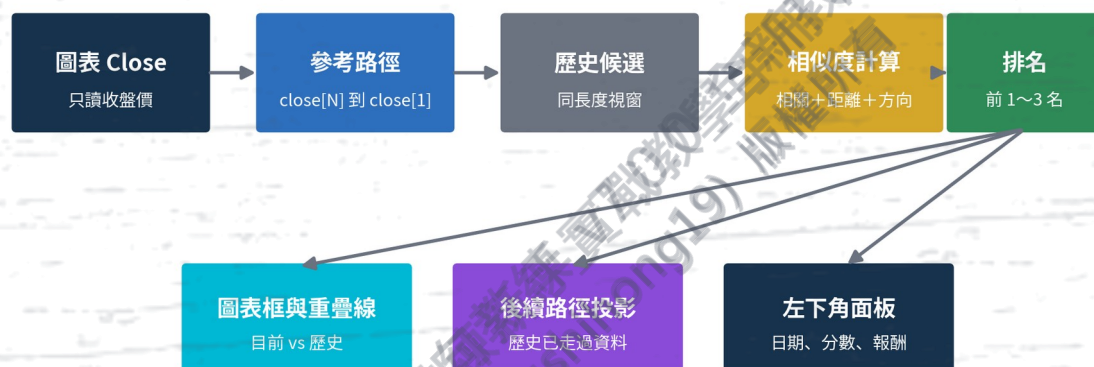
- 01 | 指標用途與整體架構
- 02 | 資料來源：為什麼只使用 Close 收盤價
- 03 | 目前參考區域：close[1] 與 close[0] 的差別
- 04 | 歷史候選區域：隔離區、不重疊與搜尋範圍
- 05 | 相似度演算法：相關、距離、方向三段式評分
- 06 | 前三名歷史樣本：門檻、排序與樣本分離
- 07 | 圖表顯示：框線、重疊線與右側歷史後續投影
- 08 | 左下角面板：每一格代表什麼
- 09 | 所有參數與預設值：哪裡可調、改了會怎樣
- 10 | 15 分鐘與 1 小時實戰教學
- 11 | 驗證位移與非重繪：怎麼做歷史盲測
- 12 | 完整實戰流程：新手如何一步一步使用

- 13 | 常見問題與故障排除
- 14 | 程式結構拆解與關鍵函式說明
- 附錄 A | 完整原始程式碼（含作者署名與授權）

01 | 指標用途與整體架構

這支指標的目的，是把「目前已收盤的連續 Close 路徑」當成樣本，與歷史上的多 Close 路徑逐段比對，出最像的歷史案例，再把那些歷史案例當時後來實際走過的路徑，重疊顯示在圖表上，助觀察：現在的路徑，過去是否曾經出現過類似節奏。

- 它不是預言器，而是歷史類比工具：重點是「找出過去最像的路徑」，再觀察歷史是怎麼延伸。
- 它不是蠟燭型態辨識器：不比較吞噬、錘頭、影線或 K 棒顏色，只比較收盤價連成的線型。
- 它不是單次訊號：每根新 K 棒收盤後，指標會以新的 `close[1]` 重新建立參考區域，再重新配對歷史。



核心原則：`close[0]` 不參與正式比對；歷史後續資料只用於驗證與投影，不回頭影響相似度。

圖 1 | 指標整體架構：參考區域 → 歷史搜尋 → 相似度計算 → 排名 → 圖表顯示

整體運作流程（你可以把它想成五步驟）

- 第一步：先決定「現在要拿哪一段線型當作樣本」。預設是最近 48 根、終點固定在 `close[1]`。
- 第二步：在歷史資料裡，依照搜尋根數與候選間隔，一段一段向更早的資料掃描。
- 第三步：每一段歷史候選都會被正規化，然後與目前樣本比對，計算三種分數。
- 第四步：低於門檻的候選直接淘汰；保留下來的候選依相似度排序，再做不重疊篩選。
- 第五步：把排名前 1~3 名成線、疊圖與右側後續投影，把重點資訊放進左下角面板。

02 | 資料來源：為什麼只使用 Close 收盤價

本指標刻意只使用 `close` 序列。這個設計看似簡單，其實是整個模型穩定性的核心。Close 收盤價是每一根 K 棒最終被市場確認的價格，它比 `high/low` 更少受到瞬間刺針影響，也比 `open` 更能代表該根 K 棒收盤時的市場共識。

- 使用 Close，可以把不同圖表類型（線圖、蠟燭圖、面積圖）統一回到同一組數值基準。
- 只用 Close，才能在數學上把路徑簡化成「一連串已確認的收盤價」，利於正規化與重疊比對。

- 若混入 high/low，候選路徑可能會被單次尖刺大幅扭曲，反而降低相似度判讀的一致性。

正常模式的時間索引



圖2 | 目前參考區域只讀取 Close；正式比對終點固定在 close[1]

這就是為什麼你切換 TradingView 的圖表類型時，指標的內部計算不會跟著改成蠟燭實體或面積圖面積。圖表外觀可以切換，但演算法的資料來源始終是 close。這種設計能確保：同樣一張圖，同樣的 K 棒資料，不會因為你把圖表從蠟燭切到線圖而改變相似度結果。

03 | 目前參考區域：close[1] 與 close[0] 的差別

這是本指標最重要的觀念之一。正常模式下，正式比對終點固定是 close[1]，不是 close[0]。close[0] 代表目前正在形成、尚未收盤的即時 K 棒。若把 close[0] 納入樣本，盤中價格每跳動一次，相似度、排名、日期與圖形就可能跟著改變，使用者會覺得結果一直漂移。

- close[1]：最新一根「已完成收盤」的 K 棒。它不會再改變，適合作為正式終點。
- close[0]：目前尚未收盤的 K 棒。價格仍在變動，若放進演算法，盤中結果會不穩定。
- 因此本指標採「收盤後更新」原則：同一根即時 K 棒內，結果保持不動；下一根 K 棒形成後才重新計算。

以預設比對長度 48 根為例

- 正常模式：參考區域為 close[48] 到 close[1]，總共 48 個資料點。
- 歷史驗證位移 = 100：參考區域改為 close[148] 到 close[101]，模擬當時只能看到 close[101] 以前的資料。
- 這也是面板會寫出「close[1] 已收盤 | close[0] 排除」或「close[101] 模擬終點」的原因。

簡單說，程式是故意保守。寧可少用一根正在形成的 K 棒，也要確保你看到的是可以重現、可以回放、可以驗證的結果，而不是盤中上下跳動的暫時畫面。

04 | 歷史候選區域：隔離區、不重疊與搜尋範圍

當目前參考區域建立完成後，程式會開始從更早的資料建立「歷史候選區域」。每一組候選的長度，必須與目前樣本完全相同。例如目前樣本是 48 根，歷史候選也一定是 48 根。這樣才是在比較兩段長度一致的路徑。

歷史候選與目前參考區域必須在日期上分離



最近候選終點 = anchorOffset + patternLength + isolationBars + futureValidationBars

「不重疊」不是不比較，而是不能共用同一批 K 棒，避免產生假高分。

圖3 | 候選區域的最近端必須離目前區域夠遠，且保留未來驗證資料

- 歷史搜尋根數：決定最遠往多久以前找樣本。預設 5000 根。
- 歷史候選間隔：決定每隔幾根建立一個候選。預設 4，能降低運算量。
- 目前區域隔離根數：避免候選後續資料貼著目前區域，造成資料洩漏或區域過近。預設 16。
- 歷史後續驗證根數：保留一段歷史「之後實際發生的走勢」，只供驗證，不參與配對。預設 16。

為了避免「明明是同一段行情，只是往前後平移幾根」卻重複入選，程式在最後排名階段要求：兩組入選歷史樣本的終點至少相隔一個完整比對長度。這就是你在面板中看到的「獨立歷史樣本」概念。

為什麼會看到「沒有任何候選達到門檻」？

- 這表示程式確實有完成歷史掃描，但所有候選的分數都低於你設定的最低相似度。
- 最常見原因是：比對長度太長、最低相似度太高，或目前市場節奏本來就不常出現。
- 處理方式通常是先把最低相似度從 55% 降到 50% 或 45%，或把比對長度從 48 改成 36。

05 | 相似度演算法：相關、距離、方向三段式評分

這支指標不直接比較絕對價格，而是先把每一段路徑轉換成「相對於第一根 Close 的百分比變化」。例如第一根作為 0%，後面每一根都換算成相對漲跌幅。這樣做的目的，是讓 30,000 美元區間與 70,000 美元區間，只要路徑比例相似，就有機會被配對成功。

三段式相似度不是單一「看起來像」



負相關不算相似；距離項防止「方向相同但幅度完全不同」；方向項保留每根 Close 的節奏。

圖 4 | 相似度由三個部分組成：相關、距離、方向

相似度評分模組總表

分數模組	權重	程式如何計算	為什麼要這樣做
Pearson 線型相關	60%	計算正規化路徑的線性相關係數，再把負相關裁成 0。	確保整段波形的高低起伏節奏相近；是整體形狀判斷的主體。
正規化路徑距離	25%	用 RMSE 比較每一個點的差距，再依波動尺度轉成 0 ~ 100 分。	避免「形狀看起來像，但幅度差很大」的路徑拿到過高分數。
方向一致率	15%	逐根比較每一段收盤是上漲、下跌或持平。	補微觀節奏，讓每一根漲跌方向越一致的樣本得到加分。

最終分數公式為：Final Similarity = Correlation Score × 0.60 + Distance Score × 0.25 + Direction Score × 0.15。分數範圍固定限制在 0 到 100。這個加權邏輯的核心思想是：先看整體形狀，再看幅度差，再看逐根方向細節。

這三種分數各自扮演什麼角色？

- 相關分數高：代表波形節奏像，例如一起上升、一起回落、一起反彈。
- 距離分數高：代表不只形狀像，連每一段偏離幅度也接近。
- 方向分數高：代表每一段收盤的漲跌方向同步率高，細節更接近。
- 只有三者一起看，才比較不會被單一高分誤導。

06 | 前三名歷史樣本：門檻、排序與樣本分離

所有候選分數算完之後，不是直接取前三名就結束。程式會先淘汰低於「最低顯示相似度」的候選，再把剩餘候選依分數由高到低檢查，符合條件者依序進入第 1、第 2、第 3 名。條件只有兩個：第一，分數夠高；第二，與已入選樣本相隔至少一個完整比對長度。

- 最低顯示相似度：預設 55.0。低於 55% 的候選，不會被顯示。
- 顯示前幾名：預設 3。你也可以改成只顯示第 1 名，讓畫面更清楚。
- 樣本分離：若 #1 與某候選的終點相隔少於 patternLength，該候選即使分數很高，也不會被當成 #2。

這就是為什麼你有時候會看到「達門檻候選很多，但最終只入選 1~2 組」：因為候選雖然多，卻可能都集中在同一段歷史附近。程式故意保守，寧可少給幾組，也不要讓三個排名其實都是同一段路徑的變形。

07 | 圖表顯示：框線、重疊線與右側歷史後續投影

目前走勢、三組歷史重疊線與右側後續投影

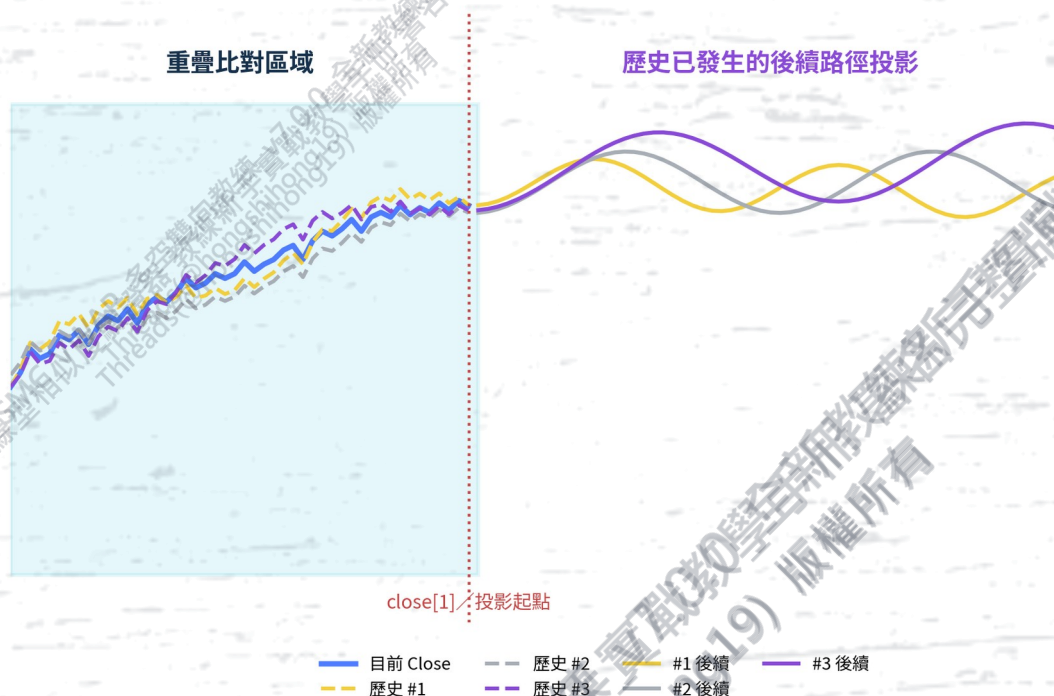
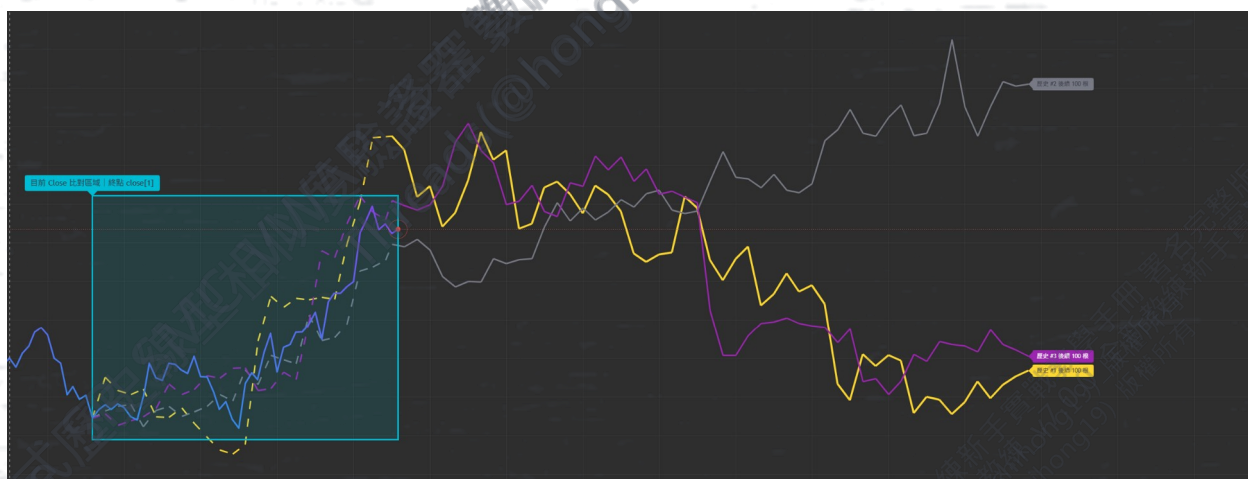


圖5 | 青色框為目前區域；黃、灰、紫框為歷史 #1~#3；右側可接出歷史後續投影



實際使用畫面 | 目前 Close 區域、三組重疊線與右側歷史後續 100 根投影

Close 歷史線型比對		v1.2.2	
模式		正常模式	
參考開始		2026/06/28 07:00	
參考結束		2026/06/30 06:00	
正式終點		close[1] 已收盤 close[0] 排除	
比對設定		48根 搜尋5000 間隔4/4 後續顯示100根	
候選統計		已測1218 達157 門檻61.0%	
更新／驗證		新 K 形成後更新 後續報酬不參與配對	
狀態		已找到 3 組獨立歷史樣本	
歷史 #1 80.9%		後續16根 -.79%	
2026/06/14 03:00 →		2026/06/16 02:00 距 340 根	
歷史 #2 80.8%		後續16根 -.49%	
2026/01/01 07:00 →		2026/01/03 06:00 距 4272 根	
歷史 #3 79.3%		後續16根 +1.29%	
2026/03/14 19:00 →		2026/03/16 18:00 距 2532 根	

實際面板 | 模式、參考日期、候選統計、前三名相似度與歷史後續報酬

圖表元素解說

圖表元素	顏色	代表的意思	判讀重點
目前參考區域	青色框	目前／驗證樣本的價格範圍，只以 Close 的最高與最低繪製。	先看這一內目前路徑的節奏。
歷史相似 #1	黃色框	分數最高的獨立歷史樣本。	通常是最值得優先觀察的案例。
歷史相似 #2	灰色框	第二高分樣本。	可以用來觀察替代情境。
歷史相似 #3	紫色框	第三高分樣本。	做多案例與空案例交錯時，第三名常能補充變化。
目前區域內重疊虛線	黃／灰／紫虛線	把歷史 #1～#3 的正規化路徑，映射到目前區域價格尺度。	可直觀看到哪一組形狀最貼近目前路徑。
歷史區域內青色虛線	青色虛線	把目前參考路徑映射回各歷史區域內。	可從歷史角度回頭比對兩段路徑。
目前右側歷史後續投影	黃／灰／紫實線	把歷史樣本後續已走過的真實路徑，按比例接在目前右側。	用來做情境推演，不是預言保證。

右側投影的數學邏輯是： $\text{Projected Y} = \text{referenceFirstClose} \times (\text{historicalFutureClose} / \text{candidateFirstClose})$ 。也就是說，歷史未來走勢不是直接照搬價格，而是保留比例關係後，重新貼到目前路徑的價格尺度。這樣做的好處，是可以把 30 美元與 70 美元的兩段歷史，放在同一個現價框架下觀察。

08 | 左下角面板：每一格代表什麼

圖面顏色與線型圖例

	青色	目前參考框／目前路徑重疊線
	黃色	歷史相似第 1 名
	灰色	歷史相似第 2 名
	紫色	歷史相似第 3 名
	紅色	排除、警示或尚未收盤

線型規則

實線：實際目前或歷史後續

虛線：正規化後的重疊比對

框：該段資料在原日期的範圍

圖 6 | 左下角面板欄位說明與顏色提示

面板欄位逐項解讀

欄位	預設顯示內容	用途	常見解讀
模式	正常模式 / 歷史驗證 位移 X	告訴目前是在看時模式還是回測模式。	若在做盲測，先確認這裡不是正常模式。
參考開始 / 參考結束	yyyy/MM/dd HH:mm	標示目前樣本的日期範圍。	可依日期拖曳圖表，直接核對歷史。
正式終點	close[1] 已收盤 close[0] 排除	告訴你現在到底用哪一根當終點。	這欄可快速排除「為什麼盤中不更新」的疑問。
比對設定	長度 搜尋 間隔 後續顯示	把目前的主要參數濃縮在一格。	改參數後先看這格是否有成功更新。
候選統計	已測 n 達 n 門檻 x%	顯示候選評估數量與達門檻數。	若已測很多但達 0，通常是門檻過高。
更新／驗證	新 K 形成後更新 後續報酬不參與配對	提示非重繪邏輯與後續報酬用途。	這欄非常適合用來提醒學員理解：投影線只是驗證材料。
狀態	已找到 n 組獨立歷史樣本 / 資料不足 / 無達門檻候選	顯示目前結果是否有效。	先看狀態，再做後續判讀。
歷史 #1 ~ #3	相似度 + 後續報酬 + 日期	濃縮每一組入選樣本的核心資訊。	先看相似度，再看後續報酬與日期。

09 | 所有參數與預設值：哪裡可調、改了會怎樣

參數名稱	預設值	可調範圍	程式如何使用	手動調整建議
目前線型比對長度	48	12 ~ 120	決定目前樣本長度。數值越大，要求整段形狀越完整；越小，越容易找到樣本。	15M 常用 36 ~ 48；1H 常用 24 ~ 48。無結果時可先降到 36。
歷史搜尋根數	5000	500 ~ 5000	往更早的資料找樣本。越大找到的案例越多，但運算也越重。	一般保留 5000 可；若商品歷史太短，可視情況下調。
歷史候選間隔	4	1 ~ 20	每隔幾根建一個候選。越小越精細，越大越省資源。	頓時可改 6 或 8；想更細緻可改 2。
目前區域隔離根數	16	1 ~ 100	讓歷史候選與目前區域保留安全距離。	想避免過近資料，可提高到 24 或 32。
歷史後續驗證根數	16	1 ~ 100	計算歷史候選完成後 N 根的實際報酬。	短線驗證可用 8 ~ 16；波段可用 24 ~ 48。

最低顯示相似度	55.0	0~100	低於此分數的候選不顯示。	沒結果就往下調；結果太雜就往上調。
顯示前幾名	3	1~3	決定要顯示幾組歷史樣本。	面太亂可改成 1 或 2。
歷史驗證位移	0	0~1000	把分析終點往歷史移動，用來做盲測。	想模擬 100 根前的當時畫面，就設 100。
歷史後續顯示根數	100	10~200	把歷史後續走勢接到目前區域右側。	畫面太擠可降到 50；想看更遠可提高到 150。
面板時間時區	Asia/Taipei	固定選項	決定面板日期的時區顯示。	常用台灣時區就維持預設。
顯示目前／驗證參考區域	開	開／關	控制青色框顯示。	教學時建議保持開啟。
顯示歷史相似區域	開	開／關	控制黃灰紫歷史框。	若只想看右側投影，可暫時關閉。
顯示重疊比對線	開	開／關	控制目前區域內的虛線疊圖。	想專心看框線時可關閉。
顯示目前右側歷史後續投影	開	開／關	控制右側黃灰紫實線投影。	預設建議開；若面過滿可關閉。

新手最常改到的三個參數

- 最低顯示相似度：如果你覺得現在怎麼都找不到相似樣本，通常先改這個。
- 目前線型比對長度：如果你覺得 48 根太長、節奏太難完全對齊，可改成 36 或 24。
- 歷史後續顯示根數：如果你只想看短線情境，不必一次投影 100 根，可以改成 40~60。

10 | 15 分鐘與 1 小時實戰教學

週期不會混用

15 分鐘圖

48 根 = 12 小時 | 短線節奏、進場附近

1 小時圖

48 根 = 48 小時 | 較大波段、方向背景

程式永遠以「目前圖表週期」的 Close 去比對同週期歷史 Close。

切換週期後，48 根代表的實際時間長度會改變，結果必須重新解讀。

圖 7 | 15 分鐘與 1 小時週期的使用差異

本指標會依照你目前開啟的圖表週期運作。15 分鐘圖比對的是 15 分鐘資料；1 小時圖比對的是 1 小時資料。它不會把 15M 與 1H 混在一起計算。

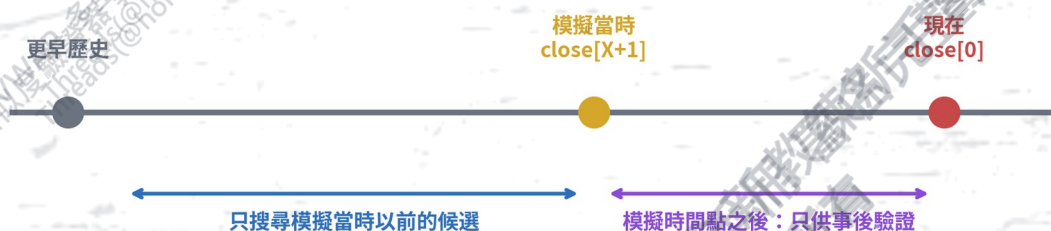
使用目標	建議週期	建議比對長度	建議門檻	說明
短線進場、日內節奏	15 分鐘	36~48	50~60%	適合觀察近 9~12 小時線型，抓日內節奏。
中短波段、大方向	1 小時	24~48	55~70%	適合觀察 1~2 天結構節奏，避免過度雜訊。

歷史盲測與教學回放	15 分鐘或 1 小時	依教學場景	維持預設 55%	配合驗證位移，示範同一套邏輯如何回看歷史。
-----------	-------------	-------	----------	-----------------------

- 若你想比較「最新 K 棒附近的細部線型」，請優先用 15 分鐘圖。
- 若你想看「近 2 天的大節奏」，可以開 1 小時圖。
- 最實用的做法：先用 1H 確認大方向，再切 15M 看目前區域與歷史是否也同向。

11 | 驗證位移與非重繪：怎麼做歷史盲測

歷史驗證位移：模擬「當時只能看到這裡」



位移 100 時，參考終點為 close[101]；close[100] 到 close[0] 禁止加入當時的配對。

圖 8 | 歷史驗證位移：把分析終點移回過去，模擬當時畫面

驗證位移是教學與回測時非常好用的功能。當你把「歷史驗證位移」設為 100，程式就會把 close[101] 當成「模擬當時最後一根已收盤 K 棒」，並禁止使用 close[100] 到 close[0] 的資料參與歷史配對。這樣你看到的配對結果，就比較接近 100 根前當下真實能看到的內容。

非重繪的關鍵設計

- 完整歷史搜尋只在 barstate.islast 執行，避免每一根歷史 K 棒都做重掃。
- 高運算量結果用 varip 快取，讓同一根即時 K 棒內只重算一次。
- 圖表框、標籤與線段會在每次更新重建，這是為了符合 Pine 的 rollback 行為。
- 正式資料固定取自 close[1] 或更早，因此同一根未收盤 K 棒中，相似度與排名不會改變。

12 | 完整實戰流程：新手如何一步一步使用

步驟 1 | 先決定你的用途。如果你是要看短線日內節奏，就開 15 分鐘圖；若要看中短期大方向，就開 1 小時圖。

步驟 2 | 確認面板「正式終點」顯示為 close[1] 已收盤 | close[0] 排除，先建立正確觀念。

步驟 3 | 觀察目前青色框內的主線型，再看黃、灰、紫虛線哪一組與目前最貼近。

步驟 4 | 讀取左下角面板的歷史 #1~#3，先看相似度，再看後續報酬與日期。

步驟 5 | 若想看同一組樣本在歷史當時如何延伸，直接看目前右側的黃、灰、紫實線投影。

步驟 6 | 若沒有任何樣本達門檻，先把最低相似度從 55% 降到 50%，或者把比對長度從 48 改成 36。

步驟 7 | 若想回頭驗證，將歷史驗證位移改成 50、100、200，模擬當時畫面，再觀察後續真實結果。

新手判讀原則

- 先看「形狀」，再看「延伸」：不要只因為某條投影看起來多，就忽略框內虛線其實跟目前主線差很多。
- 先看 #1，再看 #2、#3：第 1 名通常最值得優先參考，但當 #2、#3 都朝另一個方向時，要留意市場可能存在分歧。
- 把投影視為情境，不要當成保證：歷史相似只表示曾經出現過相似路徑，不代表未來一定複製。

13 | 常見問題與故障排除

問題	通常原因	處理方式
看不到任何歷史樣本	門檻太高或比對長度太長	先把最低相似度往下調，再視情況把長度改短。
畫面太亂	同時顯示三組歷史框、重疊線與投影線	把顯示前幾名改成 1 或 2，或關閉重疊線。
面板說歷史資料不足	商品歷史太短，或驗證位移太大	減少搜尋根數或減少驗證位移。
為什麼盤中結果不變	因為 close[0] 被排除，採收盤後更新	這是刻意設計，避免未收盤 K 棒造成結果飄移。
為什麼 #1、#2、#3 不是同一段附近	程式有做樣本分離	這是為了避免三個排名其實都來自同一段行情。
右側投影很多，但我不確定該相信哪條	歷史樣本出現多情境分歧	結合更高週期、支撐壓力或你的主交易策略做第二層確認。

14 | 程式結構拆解與關鍵函式說明

如果你會閱讀 Pine Script，本章會幫你把整份程式碼拆成容易理解的模組。即使你不寫程式，也可以用這一章快速理解：每一段程式在做什麼。

模組／函式	負責內容	為什麼重要
indicator(...)	宣告覆蓋主圖、最大 box／label／line 數量。	沒有這段，指標無法在主圖上畫框、畫線與面板。
max_bars_back(close, 5000) / max_bars_back(time, 5000)	明確配置歷史緩衝。	避免超過 5000 根造成 Pine 限制錯誤。
f_format_time()	把 time 轉成 yyyy/MM/dd HH:mm。	讓面板日期可讀。
f_signed_percent()	把報酬轉成 +x.xx% 或 -x.xx%。	面板顯示更直觀。
f_rank_color()	定義 #1 黃、#2 灰、#3 紫。	圖表與面板一致性。
f_close_bounds()	取得某一區域 Close 的最高與最低值。	畫框線高度時只使用 Close，不使用 High/Low。
f_similarity()	核心相似度計算。	把相關、距離與方向合併成最終分數。
barstate.islast 區塊	只在最後一根資料執行完整搜尋與重建圖表物件。	兼顧效能與穩定。
varip 快取陣列	儲存參考路徑、候選結果與已選樣本。	避免同一根 K 棒內重複做重運算。
目前區域右側投影線	將歷史後續真實路徑映射到現在。	把歷史結果轉成可操作的情境視圖。

關鍵變數與陣列

名稱	預設／型態	用途
patternLength	int, 預設 48	目前比對長度。
historySearchBars	int, 預設 5000	搜尋多久以前的歷史。
candidateStep /	int	設定步長與效能保護後的實際步長。

effectiveCandidateStep		
referencePath	array<float>	目前參考區域正規化後的路徑。
referenceDirections	array<int>	目前參考區域逐根方向。
candidateScores / candidateEnds	array	所有達門檻候選的分數與終點。
selectedScores / selectedEnds	array	最終入選的 #1 ~ #3。
cachedBarIndex	varip int	記住目前快取對應的是一根 bar。
drawnBoxes / drawnLabels / drawnLines	array<object>	每次重建圖表前刪除舊物件，避免累積。

結語：如何把這支指標用對

最重要的心法只有一句：先理解「它比較的是什麼」，再解讀「它暗示了什麼」。這支指標比較的是已收盤 Close 的歷史路徑相似度，並把歷史延伸情境視覺化；它不是保證行情照走，而是提供你一種高效率的歷史類比工具。當你能正確理解 close[1]、候選隔離、相似度三分法、樣本分離與右側後續投影時，這支工具就能變成你判讀節奏、講解策略、回測教學的有力輔助。

如果你要把它用在實戰，建議永遠把它放在「流程中的一環」，而不是單獨當成進出場真理。先看目前路徑與歷史是否真的相似，再看多組歷史延伸是否同向，最後再搭配你的主策略、風險管理與大週期背景做決策。這樣使用，才是最穩健、也最符合教練手冊精神的方式。

附錄 A | 完整原始程式碼

本附錄收錄可直接複製至 TradingView Pine Editor 的完整原始程式碼。程式碼已加入 Threads(@hongshihong19) 正式署名、© 2026 著作權聲明與 Mozilla Public License 2.0 授權資訊；檔案以 UTF-8 儲存，並以一般中文、英文字母、數字與標準標點編寫模組署名，降低不同平台、Word 與 PDF 轉換時產生亂碼的機率。

使用與再發布要求

- 使用、修改或再發布時，必須保留 MPL 2.0 授權聲明。
- 必須保留原作者 Threads(@hongshihong19) 的署名與原始著作權聲明。
- 禁止移除署名、冒用原作者身分，或把修改版本宣稱原作者官方版本。
- 本指標僅供技術研究、資訊整理與交易流程訓練，不構成投資建議。

```

0001 //@version=6
0002 // =====
0003 // Close 歷史線型相似度驗證器
0004 // 公開版本：v1.2.2
0005 //
0006 // 作者 / 著作權人：Threads(@hongshihong19)
0007 // © 2026 Threads(@hongshihong19) 著作權所有。
0008 //
0009 // 本 Pine Script® 原始碼依 Mozilla Public License 2.0 授權公開使用：
0010 // https://mozilla.org/MPL/2.0/
0011 //
0012 // 使用、修改或再發布本程式時，必須保留：
0013 // 1. 本授權聲明。
0014 // 2. 原作者 Threads(@hongshihong19) 的署名。
0015 // 3. 原始著作權聲明。
0016 //
0017 // 禁止移除原作者署名、冒用原作者身分，或將修改版本宣稱原作者官方版本。
0018 // 本指標僅供資訊整理、技術研究與交易流程訓練，不構成投資建議，亦不保證獲利。
0019 //
0020 // 案編碼：UTF-8。避免跨平台或 PDF 輸出亂碼，本版模組署名僅使用一般中文、
0021 // 英文字母、數字與標準標點，不使用隨機特殊符號或裝飾字元。
0022 // =====
0023 //
0024 // 版本：v1.2.2 | Threads(@hongshihong19) | 版權所有
0025 // 更新說明：加入正式作者署名與 MPL 2.0 授權版頭；統一 UTF-8 中文註解格式；
0026 // 保留 Close 歷史相似度、重疊比對、前三名樣本與目前右側後續投影功能。
0027
0028 indicator(

```

附錄 A | 完整原始程式碼（續）

```
0029 "Close 歷史線型相似度驗證器 v1.2.2 | Threads(@hongshihong19) [哥哥版]",
0030 shorttitle = "Close 線型相似度 v1.2.2",
0031 overlay = true,
0032 max_boxes_count = 10,
0033 max_labels_count = 10,
0034 max_lines_count = 500
0035 )
0036
0037 // max_bars_back() 的 num 參數在 Pine v6 只能設定 0~5000,
0038 // 本版明確配置 5000 根，將所有動態歷史索引限制在 0~4999，
0039 // 00 CE10041 與後續歷史引用越界錯誤。
0040 max_bars_back(close, 5000)
0041 max_bars_back(time, 5000)
0042
0043 // -----
0044 // [HS19] © 2026 Threads(@hongshihong19) | 模組：使用者設定
0045 // 使用者設定
0046 // -----
0047 const string GROUP_REFERENCE = "一、目前 Close 線型參考區域"
0048 const string GROUP_SEARCH = "二、歷史搜尋與相似度"
0049 const string GROUP_VALIDATION = "三、歷史驗證"
0050 const string GROUP_DISPLAY = "四、圖表與面板"
0051
0052 int patternLength = input.int(
0053     48,
0054     "目前線型比對長度",
0055     minval = 12,
0056     maxval = 120,
0057     tooltip = "正常模式使用 close[比對長度] 到 close[1]。48 根 15 分鐘 K 棒代表最近 12 小時。",
0058     group = GROUP_REFERENCE
0059 )
0060
0061 int historySearchBars = input.int(
0062     5000,
0063     "歷史搜尋根數",
0064     minval = 500,
0065     maxval = 5000,
0066     tooltip = "從參考終點往更早的歷史資料搜尋。Pine 的 max_bars_back() 上限 5000，因此本版最大設定 5000；實際可用深度還會除比對長度與驗證位移。",
0067     group = GROUP_SEARCH
0068 )
0069
0070 int candidateStep = input.int(
0071     4,
0072     "歷史候選間隔",
0073     minval = 1,
0074     maxval = 20,
0075     tooltip = "設定 4 時，每隔 4 根歷史 K 棒建立一組候選區域，可降低運算量。",
0076     group = GROUP_SEARCH
0077 )
0078
0079 int isolationBars = input.int(
0080     16,
0081     "目前區域隔離根數",
0082     minval = 1,
0083     maxval = 100,
0084     tooltip = "歷史候選的後續驗證區域與目前參考區域之間，至少保留此數量的 K 棒。",
0085     group = GROUP_SEARCH
0086 )
```

附錄 A | 完整原始程式碼（續）

```

0087
0088 int futureValidationBars = input.int(
0089     16,
0090     "歷史後續驗證根數",
0091     minval = 1,
0092     maxval = 100,
0093     tooltip = "計算歷史候選完成後第 N 根的實際報酬。此資料只供驗證，不參與相似度計算。",
0094     group = GROUP_VALIDATION
0095 )
0096
0097 float minimumSimilarity = input.float(
0098     55.0,
0099     "最低顯示相似度",
0100     minval = 0.0,
0101     maxval = 100.0,
0102     step = 0.1,
0103     tooltip = "最終相似度低於此門檻的候選不會進入排名或顯示。",
0104     group = GROUP_SEARCH
0105 )
0106
0107 int displayCount = input.int(
0108     3,
0109     "顯示前幾名",
0110     minval = 1,
0111     maxval = 3,
0112     tooltip = "顯示相似度最高且彼此不重疊的前 1~3 組歷史區域。",
0113     group = GROUP_DISPLAY
0114 )
0115
0116 int validationOffset = input.int(
0117     0,
0118     "歷史驗證位移",
0119     minval = 0,
0120     maxval = 1000,
0121     tooltip = "0：正式終點為 close[1]。100：把 close[101] 視為模擬當時最新一根已收盤 K 棒。",
0122     group = GROUP_VALIDATION
0123 )
0124
0125 int historicalFutureDisplayBars = input.int(
0126     100,
0127     "歷史後續顯示根數",
0128     minval = 10,
0129     maxval = 200,
0130     tooltip = "把每組歷史相似樣本完成後已實際發生的 Close 路徑，投影到目前參考區域右側。預設顯示約 100 根。",
0131     group = GROUP_VALIDATION
0132 )
0133
0134 string panelTimezone = input.string(
0135     "Asia/Taipei",
0136     "面板時間時區",
0137     options = {
0138         "Asia/Taipei",
0139         "Etc/UTC",
0140         "Asia/Tokyo",
0141         "America/New_York",
0142         "Europe/London"
0143     },
0144     tooltip = "日期格式固定 yyyy/MM/dd HH:mm。選項均有效 IANA 時區。",

```

附錄 A | 完整原始程式碼（續）

```

0145     group = GROUP_DISPLAY
0146 )
0147
0148 bool showCurrentBox = input.bool(
0149     true,
0150     "顯示目前／驗證參考區域",
0151     group = GROUP_DISPLAY
0152 )
0153
0154 bool showHistoricalBoxes = input.bool(
0155     true,
0156     "顯示歷史相似區域",
0157     group = GROUP_DISPLAY
0158 )
0159
0160 bool showOverlayComparison = input.bool(
0161     true,
0162     "顯示重疊比對線",
0163     tooltip = "在目前參考區域內重疊顯示歷史 #1 ~ #3 線型，並在每個歷史區域內以青色虛線重疊顯示目前 Close 路徑。",
0164     group = GROUP_DISPLAY
0165 )
0166
0167 bool showHistoricalFuturePath = input.bool(
0168     true,
0169     "顯示目前右側歷史後續投影",
0170     tooltip = "將歷史 #1 ~ #3 在配對完成後已走過的後續 Close 路徑，接在目前參考區域右側。",
0171     group = GROUP_DISPLAY
0172 )
0173
0174 // _____
0175 // [HS19] © 2026 Threads(@hongshihong19) | 模組：常數與持久物件
0176 // 常數與持久物件
0177 // _____
0178 const float EPSILON = 0.0000000001
0179 const int MAX_HISTORY_OFFSET = 4999
0180 const int PANEL_ROWS = 15
0181 const int MAX_CANDIDATE_EVALUATIONS = 1500
0182
0183 varip array<float> referencePath = array.new<float>(0)
0184 varip array<int> referenceDirections = array.new<int>(0)
0185 varip array<float> candidateScores = array.new<float>(0)
0186 varip array<int> candidateEnds = array.new<int>(0)
0187 varip array<float> selectedScores = array.new<float>(0)
0188 varip array<int> selectedEnds = array.new<int>(0)
0189
0190 varip int cachedBarIndex = na
0191 varip bool cachedReferencePathValid = false
0192 varip int cachedTestedCandidateCount = 0
0193 varip int cachedThresholdCandidateCount = 0
0194
0195 var array<box> drawnBoxes = array.new<box>(0)
0196 var array<label> drawnLabels = array.new<label>(0)
0197 var array<line> drawnLines = array.new<line>(0)
0198
0199 var table resultPanel = table.new(
0200     position.bottom_left,
0201     2,
0202     PANEL_ROWS,

```

附錄 A | 完整原始程式碼 (續)

```

0203 border_width = 1,
0204 frame_width = 1,
0205 frame_color = color.new(color.gray, 40),
0206 border_color = color.new(color.gray, 65)
0207 )
0208
0209
0210 //
0211 // [HS19] © 2026 Threads(@hongshihong19) | 模組：共用函式與相似度核心
0212 // 工具函式
0213 //
0214 f format_time(int timestampValue, string timezoneValue) =>
0215     string result = "資料不足"
0216     if not na(timestampValue)
0217         result := str.format_time(timestampValue, "yyyy/MM/dd HH:mm", timezoneValue)
0218     result
0219
0220 f signed_percent(float value) =>
0221     string result = "資料不足"
0222     if not na(value)
0223         string prefix = value > 0.0 ? "+" : ""
0224         result := prefix + str.tostring(value, "#.00") + "%"
0225     result
0226
0227 f rank_color(int rankIndex) =>
0228     color result = color.purple
0229     if rankIndex == 0
0230         result := color.yellow
0231     else if rankIndex == 1
0232         result := color.gray
0233     result
0234
0235 f close_bounds(int endOffset, int lengthValue) =>
0236     float highestClose = na
0237     float lowestClose = na
0238     for i = 0 to lengthValue - 1
0239         int historicalOffset = endOffset + i
0240         float value = close[historicalOffset]
0241         if not na(value)
0242             highestClose := na(highestClose) ? value : math.max(highestClose, value)
0243             lowestClose := na(lowestClose) ? value : math.min(lowestClose, value)
0244     [highestClose, lowestClose]
0245
0246 f similarity(
0247     int candidateEndOffset,
0248     int lengthValue,
0249     array<float> refPath,
0250     array<int> refDirections,
0251     float refSum,
0252     float refSumSquares,
0253     float refStandardDeviation
0254 ) =>
0255     float result = na
0256     int candidateOldestOffset = candidateEndOffset + lengthValue - 1
0257     float candidateFirstClose = close[candidateOldestOffset]
0258
0259     if not na(candidateFirstClose) and math.abs(candidateFirstClose) > EPSILON and array.size(refPath) == lengthValue
0260         float candidateSum = 0.0

```

附錄 A | 完整原始程式碼（續）

```

0261 float candidateSumSquares = 0.0
0262 float crossSum = 0.0
0263 float squaredDifferenceSum = 0.0
0264 int directionMatches = 0
0265 bool validCandidate = true
0266
0267 for i = 0 to lengthValue - 1
0268   int candidateOffset = candidateOldestOffset - i
0269   float candidateClose = close[candidateOffset]
0270   float referenceNormalized = array.get(refPath, i)
0271
0272   if na(candidateClose) or na(referenceNormalized)
0273     validCandidate := false
0274   else
0275     float candidateNormalized = (candidateClose / candidateFirstClose - 1.0) * 100.0
0276     candidateSum += candidateNormalized
0277     candidateSumSquares += candidateNormalized * candidateNormalized
0278     crossSum += referenceNormalized * candidateNormalized
0279
0280   float pointDifference = referenceNormalized - candidateNormalized
0281   squaredDifferenceSum += pointDifference * pointDifference
0282
0283   if i > 0
0284     float candidatePreviousClose = close[candidateOffset + 1]
0285     if na(candidatePreviousClose)
0286       validCandidate := false
0287     else
0288       int candidateDirection = 0
0289       if candidateClose > candidatePreviousClose
0290         candidateDirection := 1
0291       else if candidateClose < candidatePreviousClose
0292         candidateDirection := -1
0293
0294     int referenceDirection = array.get(refDirections, i - 1)
0295     if candidateDirection == referenceDirection
0296       directionMatches += 1
0297
0298   if validCandidate
0299     float sampleCount = float(lengthValue)
0300     float referenceVarianceNumerator = math.max(
0301       sampleCount * refSumSquares - refSum * refSum,
0302       0.0
0303     )
0304     float candidateVarianceNumerator = math.max(
0305       sampleCount * candidateSumSquares - candidateSum * candidateSum,
0306       0.0
0307     )
0308     float covarianceNumerator = sampleCount * crossSum - refSum * candidateSum
0309     float correlationDenominator = math.sqrt(
0310       referenceVarianceNumerator * candidateVarianceNumerator
0311     )
0312
0313     float correlation = 0.0
0314     if referenceVarianceNumerator <= EPSILON and candidateVarianceNumerator <= EPSILON
0315       correlation := squaredDifferenceSum <= EPSILON ? 1.0 : 0.0
0316     else if correlationDenominator > EPSILON
0317       correlation := covarianceNumerator / correlationDenominator
0318

```

附錄 A | 完整原始程式碼（續）

```

0319 correlation := math.max(-1.0, math.min(1.0, correlation))
0320 float correlationScore = math.max(correlation, 0.0) * 100.0
0321
0322 float rmse = math.sqrt(squaredDifferenceSum / sampleCount)
0323 float candidateMean = candidateSum / sampleCount
0324 float candidateVariance = math.max(
0325     candidateSumSquares / sampleCount - candidateMean * candidateMean,
0326     0.0
0327 )
0328 float candidateStandardDeviation = math.sqrt(candidateVariance)
0329 float distanceScale = math.max(
0330     (refStandardDeviation + candidateStandardDeviation) * 0.5,
0331     0.05
0332 )
0333 float distanceScore = 100.0 / (1.0 + rmse / distanceScale)
0334
0335 float directionScore = 100.0
0336 if lengthValue > 1
0337     directionScore := float(directionMatches) / float(lengthValue - 1) * 100.0
0338
0339 float finalScore = (
0340     correlationScore * 0.60 +
0341     distanceScore * 0.25 +
0342     directionScore * 0.15
0343 )
0344 result := math.max(0.0, math.min(100.0, finalScore))
0345
0346 result
0347
0348 //
0349 // [HS19] © 2026 Threads(@hongshihong19) | 模組：歷史搜尋、排序與非重繪快取
0350 // 完整搜尋只在圖表最後一根資料執行。
0351 // 指標在即時 K 棒的每次資料更新會重新執行，但所有配對資料固定取自 close[1]
0352 // 或更早的已收盤資料，因此同一根即時 K 棒內的相似度、日期及排名不會變動。
0353 // 每次執行仍重建圖表物件，確保 Pine 時回滾後線與標籤持續顯示。
0354 //
0355 if barstate.islast
0356
0357     // 初始化 box / label 累積。
0358     // 這段在每次即時更新執行，確保 Pine rollback 後仍能重建目前顯示。
0359     if array.size(drawnBoxes) > 0
0360         for objectIndex = 0 to array.size(drawnBoxes) - 1
0361             box.delete(array.get(drawnBoxes, objectIndex))
0362         array.clear(drawnBoxes)
0363
0364     if array.size(drawnLabels) > 0
0365         for objectIndex = 0 to array.size(drawnLabels) - 1
0366             label.delete(array.get(drawnLabels, objectIndex))
0367         array.clear(drawnLabels)
0368
0369     if array.size(drawnLines) > 0
0370         for objectIndex = 0 to array.size(drawnLines) - 1
0371             line.delete(array.get(drawnLines, objectIndex))
0372         array.clear(drawnLines)
0373
0374     // 空面板有儲存格；面板每次時更新只做輕量重繪。
0375     for panelRow = 0 to PANEL_ROWS - 1
0376         table.cell(

```

附錄 A | 完整原始程式碼（續）

```

0377     resultPanel,
0378     0,
0379     panelRow,
0380     "",
0381     text_color = color.white,
0382     bgcolor = color.new(color.black, 100),
0383     text_halign = text.align_left,
0384     text_size = size.small
0385 )
0386 table.cell(
0387     resultPanel,
0388     1,
0389     panelRow,
0390     "",
0391     text_color = color.white,
0392     bgcolor = color.new(color.black, 100),
0393     text_halign = text.align_left,
0394     text_size = size.small
0395 )
0396
0397 // validationOffset = 0 時，anchorOffset = 1，即正式終點 close[1].
0398 int anchorOffset = validationOffset + 1
0399 int anchorOldestOffset = anchorOffset + patternLength - 1
0400
0401 // 歷史候選的最近端：
0402 // candidateEnd - futureValidationBars 與 anchorOldestOffset 之間至少留 isolationBars 根。
0403 // [] candidateEnd >= anchorOffset + patternLength + isolationBars + futureValidationBars.
0404 int minimumCandidateEnd = (
0405     anchorOffset +
0406     patternLength +
0407     isolationBars +
0408     futureValidationBars
0409 )
0410
0411 // 歷史搜尋的最遠端同時受：
0412 // 1. 使用者設定；2. 圖表實際 bar_index；3. 5000 根歷史緩衝上限約束。
0413 int requestedMaximumCandidateEnd = anchorOffset + historySearchBars
0414 int loadedMaximumCandidateEnd = bar_index - (patternLength - 1)
0415 int bufferMaximumCandidateEnd = MAX_HISTORY_OFFSET - (patternLength - 1)
0416 int maximumCandidateEnd = math.min(
0417     requestedMaximumCandidateEnd,
0418     math.min(loadedMaximumCandidateEnd, bufferMaximumCandidateEnd)
0419 )
0420
0421 // 極端設定（例如搜尋 5000 根、間隔 1、長度 120）可能觸發 Pine 單次圈
0422 // 執行時間限制。程式最多評估約 1500 組候選；只有必要時才自動放大實際間隔。
0423 // 此步長只由 input 值計算，不依賴每根 K 棒變動的 series 值。
0424 int configuredCandidateSpan = math.max(
0425     historySearchBars -
0426     (patternLength + isolationBars + futureValidationBars) +
0427     1,
0428     1
0429 )
0430 int performanceRequiredStep = int(
0431     math.ceil(
0432         float(configuredCandidateSpan) /
0433         float(MAX_CANDIDATE_EVALUATIONS)
0434     )

```

附錄 A | 完整原始程式碼（續）

```

0435 )
0436 int effectiveCandidateStep = math.max(
0437     candidateStep,
0438     performanceRequiredStep
0439 )
0440
0441 bool referenceDataAvailable = (
0442     anchorOldestOffset <= bar_index and
0443     anchorOldestOffset <= MAX_HISTORY_OFFSET and
0444     not na(close[anchorOldestOffset]) and
0445     not na(close[anchorOffset])
0446 )
0447
0448 bool searchRangeAvailable = (
0449     referenceDataAvailable and
0450     maximumCandidateEnd >= minimumCandidateEnd
0451 )
0452
0453 // varip 快取讓高計算量搜尋在同一根時 K 棒只執行一次。
0454 // 框線、標籤與表格仍會在每次即時更新重建，以符合 Pine rollback 行。
0455 bool shouldRunSearch = (
0456     na(cachedBarIndex) or
0457     cachedBarIndex != bar_index
0458 )
0459
0460 if shouldRunSearch
0461     cachedBarIndex := bar_index
0462     cachedReferencePathValid := referenceDataAvailable
0463     cachedTestedCandidateCount := 0
0464     cachedThresholdCandidateCount := 0
0465
0466     array.clear(referencePath)
0467     array.clear(referenceDirections)
0468     array.clear(candidateScores)
0469     array.clear(candidateEnds)
0470     array.clear(selectedScores)
0471     array.clear(selectedEnds)
0472
0473     float referenceSum = 0.0
0474     float referenceSumSquares = 0.0
0475     float referenceStandardDeviation = na
0476
0477     // 建立目前／驗證參考區域的正規化 Close 路徑。
0478     if referenceDataAvailable
0479         float referenceFirstClose = close[anchorOldestOffset]
0480         if na(referenceFirstClose) or math.abs(referenceFirstClose) <= EPSILON
0481             cachedReferencePathValid := false
0482         else
0483             for i = 0 to patternLength - 1
0484                 int referenceOffset = anchorOldestOffset - i
0485                 float referenceClose = close[referenceOffset]
0486
0487                 if na(referenceClose)
0488                     cachedReferencePathValid := false
0489                 else
0490                     float referenceNormalized = (
0491                         referenceClose / referenceFirstClose - 1.0
0492                     ) * 100.0

```

附錄 A | 完整原始程式碼（續）

```

0493         array.push(referencePath, referenceNormalized)
0494         referenceSum += referenceNormalized
0495         referenceSumSquares += (
0496             referenceNormalized *
0497             referenceNormalized
0498         )
0499     }
0500
0501     if i > 0
0502         float previousReferenceClose = close[referenceOffset + 1]
0503         if na(previousReferenceClose)
0504             cachedReferencePathValid := false
0505         else
0506             int referenceDirection = 0
0507             if referenceClose > previousReferenceClose
0508                 referenceDirection := 1
0509             else if referenceClose < previousReferenceClose
0510                 referenceDirection := -1
0511             array.push(
0512                 referenceDirections,
0513                 referenceDirection
0514             )
0515
0516     if (
0517         cachedReferencePathValid and
0518         array.size(referencePath) == patternLength and
0519         array.size(referenceDirections) == patternLength - 1
0520     )
0521         float referenceCount = float(patternLength)
0522         float referenceMean = referenceSum / referenceCount
0523         float referenceVariance = math.max(
0524             referenceSumSquares / referenceCount -
0525             referenceMean * referenceMean,
0526             0.0
0527         )
0528         referenceStandardDeviation := math.sqrt(
0529             referenceVariance
0530         )
0531     else
0532         cachedReferencePathValid := false
0533
0534     // 搜尋歷史候選。歷史後續資料不傳入 f_similarity0，不參與配對分數。
0535     if searchRangeAvailable and cachedReferencePathValid
0536         for candidateEnd = minimumCandidateEnd to maximumCandidateEnd by effectiveCandidateStep
0537             cachedTestedCandidateCount += 1
0538
0539         float similarity = f_similarity(
0540             candidateEnd,
0541             patternLength,
0542             referencePath,
0543             referenceDirections,
0544             referenceSum,
0545             referenceSumSquares,
0546             referenceStandardDeviation
0547         )
0548
0549     if (
0550         not na(similarity) and

```

附錄 A | 完整原始程式碼（續）

```

0551         similarity >= minimumSimilarity
0552     )
0553     array.push(candidateScores, similarity)
0554     array.push(candidateEnds, candidateEnd)
0555     cachedThresholdCandidateCount += 1
0556
0557     // 依相似度由高至低選取前 1~3 名。
0558     // 入選樣本的終點至少相隔 patternLength，避免同一段行情重複入選
0559     if array.size(candidateScores) > 0
0560         for rankIndex = 0 to displayCount - 1
0561             float bestScore = na
0562             int bestEnd = na
0563
0564             for candidateIndex = 0 to array.size(candidateScores) - 1
0565                 float score = array.get(
0566                     candidateScores,
0567                     candidateIndex
0568                 )
0569                 int candidateEnd = array.get(
0570                     candidateEnds,
0571                     candidateIndex
0572                 )
0573                 bool sufficientlySeparated = true
0574
0575                 if array.size(selectedEnds) > 0
0576                     for selectedIndex = 0 to array.size(selectedEnds) - 1
0577                         int existingEnd = array.get(
0578                             selectedEnds,
0579                             selectedIndex
0580                         )
0581                         if (
0582                             math.abs(candidateEnd - existingEnd) <
0583                             patternLength
0584                         )
0585                             sufficientlySeparated := false
0586
0587                 if (
0588                     sufficientlySeparated and
0589                     (na(bestScore) or score > bestScore)
0590                 )
0591                     bestScore := score
0592                     bestEnd := candidateEnd
0593
0594             if not na(bestScore) and not na(bestEnd)
0595                 array.push(selectedScores, bestScore)
0596                 array.push(selectedEnds, bestEnd)
0597
0598     //
0599     // [HS19] © 2026 Threads(@hongshihong19) | 模組：目前區域與歷史重疊線
0600     // 圖表框選：目前 / 驗證參考區域
0601     //
0602     float referenceFirstClose = cachedReferencePathValid ? close[anchorOldestOffset] : na
0603     int overlaySegmentStep = math.max(1, int(math.ceil(float(math.max(patternLength - 1, 1) * math.max(displayCount, 1) * 2) / 180.0)))
0604     int futureProjectionStep = math.max(1, int(math.ceil(float(math.max(historicalFutureDisplayBars, 1) * math.max(displayCount, 1)) / 180.0)))
0605
0606     if cachedReferencePathValid and showCurrentBox
0607         [referenceTop, referenceBottom] = f_close_bounds(
0608             anchorOffset,

```

附錄 A | 完整原始程式碼（續）

```

0609     patternLength
0610 )
0611
0612 if not na(referenceTop) and not na(referenceBottom)
0613     float referenceRange = referenceTop - referenceBottom
0614     float referencePadding = math.max(
0615         referenceRange * 0.05,
0616         math.max(
0617             math.abs(close[anchorOffset]) * 0.0001,
0618             syminfo.mintick * 10.0
0619         )
0620     )
0621
0622     int referenceLeftTime = time[anchorOldestOffset]
0623     int referenceRightTime = time[anchorOffset - 1]
0624     color referenceColor = color.aqua
0625
0626     box referenceBox = box.new(
0627         left = referenceLeftTime,
0628         top = referenceTop + referencePadding,
0629         right = referenceRightTime,
0630         bottom = referenceBottom - referencePadding,
0631         xloc = xloc.bar_time,
0632         border_color = referenceColor,
0633         border_width = 2,
0634         bgcolor = color.new(referenceColor, 88)
0635     )
0636     array.push(drawnBoxes, referenceBox)
0637
0638     string referenceLabelText = "目前 Close 比對區域 | 終點 close[1]"
0639     if validationOffset > 0
0640         referenceLabelText := (
0641             "歷史驗證區域 | 位移" +
0642             str.tostring(validationOffset) +
0643             " 根 | 終點 close[" +
0644             str.tostring(anchorOffset) +
0645             "]"
0646         )
0647
0648     label referenceLabel = label.new(
0649         x = referenceLeftTime,
0650         y = referenceTop + referencePadding,
0651         text = referenceLabelText,
0652         xloc = xloc.bar_time,
0653         yloc = yloc.price,
0654         style = label.style_label_down,
0655         color = referenceColor,
0656         textcolor = color.black,
0657         size = size.small
0658     )
0659     array.push(drawnLabels, referenceLabel)
0660
0661     // 在目前參考區域內，將歷史 #1~#3 的正規化 Close 路徑轉換到目前價格尺度，
0662     // 以虛線重疊顯示，方便直接與目前主線型比較。
0663     if showOverlayComparison and array.size(selectedEnds) > 0 and not na(referenceFirstClose) and math.abs(referenceFirstClose) > EPSILON
0664         for rankIndex = 0 to array.size(selectedEnds) - 1
0665             int candidateEnd = array.get(selectedEnds, rankIndex)
0666             int candidateOldestOffset = candidateEnd + patternLength - 1

```

附錄 A | 完整原始程式碼（續）

```

0667         float candidateFirstClose = close[candidateOldestOffset]
0668         color overlayColor = f_rank_color(rankIndex)
0669
0670         if not na(candidateFirstClose) and math.abs(candidateFirstClose) > EPSILON
0671             for pointIndex = 0 to patternLength - 2 by overlaySegmentStep
0672                 int nextPointIndex = math.min(pointIndex + overlaySegmentStep, patternLength - 1)
0673                 float candidateClose1 = close[candidateOldestOffset - pointIndex]
0674                 float candidateClose2 = close[candidateOldestOffset - nextPointIndex]
0675
0676                 if not na(candidateClose1) and not na(candidateClose2)
0677                     float candidateNorm1 = (candidateClose1 / candidateFirstClose - 1.0) * 100.0
0678                     float candidateNorm2 = (candidateClose2 / candidateFirstClose - 1.0) * 100.0
0679                     float overlayY1 = referenceFirstClose * (1.0 + candidateNorm1 / 100.0)
0680                     float overlayY2 = referenceFirstClose * (1.0 + candidateNorm2 / 100.0)
0681                     int overlayX1 = time[anchorOldestOffset - pointIndex]
0682                     int overlayX2 = time[anchorOldestOffset - nextPointIndex]
0683
0684                     line currentCompareLine = line.new(
0685                         x1 = overlayX1,
0686                         y1 = overlayY1,
0687                         x2 = overlayX2,
0688                         y2 = overlayY2,
0689                         xloc = xloc.bar_time,
0690                         extend = extend.none,
0691                         color = overlayColor,
0692                         style = line.style_dashed,
0693                         width = 2
0694                     )
0695                     array.push(drawnLines, currentCompareLine)
0696
0697 // -----
0698 // [HS19] © 2026 Threads(@hongshihong19) | 模組：歷史後續路徑投影
0699 // 目前參考區域右側：歷史 #1 ~ #3 後續實際走勢投影
0700 // 每條投影線使用該歷史樣本「配對區域第一根 Close」作比例基準，
0701 // 因此會與目前框內同色虛線的最後一點無縫銜接。
0702 // 正常模式最多向目前 K 棒右側投影 200 根，低於 Pine 的 500 根未來繪圖限制。
0703 // -----
0704 if showHistoricalFuturePath and cachedReferencePathValid and array.size(selectedEnds) > 0 and not na(referenceFirstClose) and math.abs(referenceFirstClose) >
EPSILON
0705     int referenceEndBarIndex = bar_index - anchorOffset
0706
0707     for rankIndex = 0 to array.size(selectedEnds) - 1
0708         int candidateEnd = array.get(selectedEnds, rankIndex)
0709         int candidateOldestOffset = candidateEnd + patternLength - 1
0710         float candidateFirstClose = close[candidateOldestOffset]
0711         color projectionColor = f_rank_color(rankIndex)
0712         int availableFutureBars = math.min(historicalFutureDisplayBars, candidateEnd - 1)
0713
0714         if not na(candidateFirstClose) and math.abs(candidateFirstClose) > EPSILON and availableFutureBars > 0
0715             for futureIndex = 0 to availableFutureBars - 1 by futureProjectionStep
0716                 int nextFutureIndex = math.min(futureIndex + futureProjectionStep, availableFutureBars)
0717                 int futureOffset1 = candidateEnd - futureIndex
0718                 int futureOffset2 = candidateEnd - nextFutureIndex
0719                 float historicalFutureClose1 = close[futureOffset1]
0720                 float historicalFutureClose2 = close[futureOffset2]
0721
0722                 if not na(historicalFutureClose1) and not na(historicalFutureClose2)
0723                     float projectedY1 = referenceFirstClose * (historicalFutureClose1 / candidateFirstClose)
0724                     float projectedY2 = referenceFirstClose * (historicalFutureClose2 / candidateFirstClose)

```

附錄 A | 完整原始程式碼（續）

```

0725         int projectedX1 = referenceEndBarIndex + futureIndex
0726         int projectedX2 = referenceEndBarIndex + nextFutureIndex
0727
0728         line projectedFutureLine = line.new(
0729             x1 = projectedX1,
0730             y1 = projectedY1,
0731             x2 = projectedX2,
0732             y2 = projectedY2,
0733             xloc = xloc.bar_index,
0734             extend = extend.none,
0735             color = projectionColor,
0736             style = line.style_solid,
0737             width = rankIndex == 0 ? 3 : 2
0738         )
0739         array.push(drawnLines, projectedFutureLine)
0740
0741         int finalFutureOffset = candidateEnd - availableFutureBars
0742         float finalHistoricalClose = close[finalFutureOffset]
0743         if not na(finalHistoricalClose)
0744             float finalProjectedY = referenceFirstClose * (finalHistoricalClose / candidateFirstClose)
0745             int finalProjectedX = referenceEndBarIndex + availableFutureBars
0746             string projectionLabelText = (
0747                 "歷史 #" + str.toString(rankIndex + 1) +
0748                 " 後續 " + str.toString(availableFutureBars) + " 根"
0749             )
0750
0751             label projectionLabel = label.new(
0752                 x = finalProjectedX,
0753                 y = finalProjectedY,
0754                 text = projectionLabelText,
0755                 xloc = xloc.bar_index,
0756                 yloc = yloc.price,
0757                 style = label.style_label_left,
0758                 color = projectionColor,
0759                 textcolor = rankIndex == 2 ? color.white : color.black,
0760                 size = size.tiny
0761             )
0762             array.push(drawnLabels, projectionLabel)
0763
0764             //
0765             // 圖表框選：歷史相似區域
0766             //
0767             if showHistoricalBoxes and array.size(selectedEnds) > 0
0768                 for rankIndex = 0 to array.size(selectedEnds) - 1
0769                     int candidateEnd = array.get(selectedEnds, rankIndex)
0770                     float similarity = array.get(selectedScores, rankIndex)
0771                     int candidateOldestOffset = candidateEnd + patternLength - 1
0772
0773                     [candidateTop, candidateBottom] = f_close_bounds(
0774                         candidateEnd,
0775                         patternLength
0776                     )
0777
0778                     if not na(candidateTop) and not na(candidateBottom)
0779                         float candidateRange = candidateTop - candidateBottom
0780                         float candidatePadding = math.max(
0781                             candidateRange * 0.05,
0782                             math.max(

```

附錄 A | 完整原始程式碼（續）

```

0783         math.abs(close[candidateEnd]) * 0.0001,
0784         syminfo.mintick * 10.0
0785     )
0786 )
0787
0788 int candidateLeftTime = time[candidateOldestOffset]
0789 int candidateRightTime = time[candidateEnd - 1]
0790 color candidateColor = f_rank_color(rankIndex)
0791
0792 box candidateBox = box.new(
0793     left = candidateLeftTime,
0794     top = candidateTop + candidatePadding,
0795     right = candidateRightTime,
0796     bottom = candidateBottom - candidatePadding,
0797     xloc = xloc.bar_time,
0798     border_color = candidateColor,
0799     border_width = 2,
0800     bgcolor = color.new(candidateColor, 88)
0801 )
0802 array.push(drawnBoxes, candidateBox)
0803
0804 string candidateLabelText = (
0805     "歷史相似 #" +
0806     str.tostring(rankIndex + 1) +
0807     " | " +
0808     str.tostring(similarity, "#.0") +
0809     "%"
0810 )
0811
0812 label candidateLabel = label.new(
0813     x = candidateLeftTime,
0814     y = candidateTop + candidatePadding,
0815     text = candidateLabelText,
0816     xloc = xloc.bar_time,
0817     yloc = yloc.price,
0818     style = label.style_label_down,
0819     color = candidateColor,
0820     textcolor = rankIndex == 2 ? color.white : color.black,
0821     size = size.small
0822 )
0823 array.push(drawnLabels, candidateLabel)
0824
0825 // 在每個歷史相似區域內，以青色虛線重疊顯示目前參考區域的正規化 Close 路徑。
0826 if showOverlayComparison and cachedReferencePathValid and not na(referenceFirstClose) and math.abs(referenceFirstClose) > EPSILON
0827     float candidateFirstClose = close[candidateOldestOffset]
0828     if not na(candidateFirstClose) and math.abs(candidateFirstClose) > EPSILON
0829         for pointIndex = 0 to patternLength - 2 by overlaySegmentStep
0830             int nextPointIndex = math.min(pointIndex + overlaySegmentStep, patternLength - 1)
0831             float refNorm1 = array.get(referencePath, pointIndex)
0832             float refNorm2 = array.get(referencePath, nextPointIndex)
0833             float overlayY1 = candidateFirstClose * (1.0 + refNorm1 / 100.0)
0834             float overlayY2 = candidateFirstClose * (1.0 + refNorm2 / 100.0)
0835             int overlayX1 = time[candidateOldestOffset - pointIndex]
0836             int overlayX2 = time[candidateOldestOffset - nextPointIndex]
0837
0838             line historyCompareLine = line.new(
0839                 x1 = overlayX1,
0840                 y1 = overlayY1,

```

附錄 A | 完整原始程式碼（續）

```

0841         x2 = overlayX2,
0842         y2 = overlayY2,
0843         xloc = xloc.bar_time,
0844         extend = extend.none,
0845         color = color.aqua,
0846         style = line.style_dashed,
0847         width = 2
0848     )
0849     array.push(drawnLines, historyCompareLine)
0850
0851
0852 // -----
0853 // [HS19] © 2026 Threads(@hongshihong19) | 模組：左下角結果面板
0854 // 左下角精簡面板
0855 // 固定使用 15 列，不再建立 40 列空白儲存格，避免一般視窗必須切換全螢幕
0856 // 才能看完整面板。每個歷史樣本只使用 2 列，仍保留日期、相似度、距離
0857 // 與歷史後續報酬。
0858 // -----
0859 color titleBackground = color.new(color.blue, 15)
0860 color normalBackground = color.new(color.black, 20)
0861 color warningBackground = color.new(color.orange, 70)
0862 color errorBackground = color.new(color.red, 40)
0863 color valueTextColor = color.white
0864
0865 table.cell(
0866     resultPanel,
0867     0,
0868     0,
0869     "Close 歷史線型比對",
0870     text_color = color.white,
0871     bgcolor = titleBackground,
0872     text_halign = text.align_left,
0873     text_size = size.small
0874 )
0875 table.cell(
0876     resultPanel,
0877     1,
0878     0,
0879     "v1.2.2",
0880     text_color = color.white,
0881     bgcolor = titleBackground,
0882     text_halign = text.align_right,
0883     text_size = size.small
0884 )
0885
0886 string modeText = validationOffset == 0 ? "正常模式" : "歷史驗證 | 位移" + str.toString(validationOffset)
0887 table.cell(resultPanel, 0, 1, "模式", text_color = color.white, bgcolor = normalBackground, text_halign = text.align_left, text_size = size.tiny)
0888 table.cell(resultPanel, 1, 1, modeText, text_color = valueTextColor, bgcolor = normalBackground, text_halign = text.align_left, text_size = size.tiny)
0889
0890 string referenceStartText = referenceDataAvailable ? f_format_time(time[anchorOldestOffset], panelTimezone) : "資料不足"
0891 string referenceEndText = referenceDataAvailable ? f_format_time(time[anchorOffset], panelTimezone) : "資料不足"
0892
0893 table.cell(resultPanel, 0, 2, "參考開始", text_color = color.white, bgcolor = normalBackground, text_halign = text.align_left, text_size = size.tiny)
0894 table.cell(resultPanel, 1, 2, referenceStartText, text_color = valueTextColor, bgcolor = normalBackground, text_halign = text.align_left, text_size = size.tiny)
0895
0896 table.cell(resultPanel, 0, 3, "參考結束", text_color = color.white, bgcolor = normalBackground, text_halign = text.align_left, text_size = size.tiny)
0897 table.cell(resultPanel, 1, 3, referenceEndText, text_color = valueTextColor, bgcolor = normalBackground, text_halign = text.align_left, text_size = size.tiny)
0898

```

附錄 A | 完整原始程式碼（續）

```

0899 string endpointText = "close[1] 已收盤 | close[0] 排除"
0900 if validationOffset > 0
0901     endpointText := "close[" + str.toString(anchorOffset) + "]" 模擬終點 | 其後資料排除"
0902
0903 table.cell(resultPanel, 0, 4, "正式終點", text_color = color.white, bgcolor = warningBackground, text_halign = text.align_left, text_size = size.tiny)
0904 table.cell(resultPanel, 1, 4, endpointText, text_color = color.white, bgcolor = warningBackground, text_halign = text.align_left, text_size = size.tiny)
0905
0906 string compareSettingText = (
0907     str.toString(patternLength) + "根 | 搜尋" +
0908     str.toString(historySearchBars) + " | 間隔" +
0909     str.toString(candidateStep) + "/" +
0910     str.toString(effectiveCandidateStep) + " | 後續顯示" +
0911     str.toString(historicalFutureDisplayBars) + "根"
0912 )
0913 table.cell(resultPanel, 0, 5, "比對設定", text_color = color.white, bgcolor = normalBackground, text_halign = text.align_left, text_size = size.tiny)
0914 table.cell(resultPanel, 1, 5, compareSettingText, text_color = valueTextColor, bgcolor = normalBackground, text_halign = text.align_left, text_size = size.tiny)
0915
0916 string candidateSummaryText = (
0917     "已測" + str.toString(cachedTestedCandidateCount) +
0918     " | 達" + str.toString(cachedThresholdCandidateCount) +
0919     " | 門檻" + str.toString(minimumSimilarity, "#.0") + "%"
0920 )
0921 table.cell(resultPanel, 0, 6, "候選統計", text_color = color.white, bgcolor = normalBackground, text_halign = text.align_left, text_size = size.tiny)
0922 table.cell(resultPanel, 1, 6, candidateSummaryText, text_color = valueTextColor, bgcolor = normalBackground, text_halign = text.align_left, text_size = size.tiny)
0923
0924 table.cell(resultPanel, 0, 7, "更新 / 驗證", text_color = color.white, bgcolor = warningBackground, text_halign = text.align_left, text_size = size.tiny)
0925 table.cell(resultPanel, 1, 7, "新 K 形成後更新 | 後續報酬不參與配對", text_color = color.white, bgcolor = warningBackground, text_halign = text.align_left, text_size = size.tiny)
0926
0927 string statusText = ""
0928 color statusBackground = normalBackground
0929 if not referenceDataAvailable or not cachedReferencePathValid
0930     statusText := "參考區域資料不足，無法建立 Close 路徑"
0931     statusBackground := errorBackground
0932 else if not searchRangeAvailable
0933     statusText := "歷史資料不足，無法完成比對"
0934     statusBackground := errorBackground
0935 else if array.size(selectedEnds) == 0
0936     statusBackground := warningBackground
0937     if cachedThresholdCandidateCount == 0
0938         statusText := "沒有任何候選達到 " + str.toString(minimumSimilarity, "#.0") + "%"
0939     else
0940         statusText := "達門檻候選經樣本分離後無有效排名"
0941 else
0942     statusText := "已找到 " + str.toString(array.size(selectedEnds)) + " 組獨立歷史樣本"
0943
0944 table.cell(resultPanel, 0, 8, "狀態", text_color = color.white, bgcolor = statusBackground, text_halign = text.align_left, text_size = size.tiny)
0945 table.cell(resultPanel, 1, 8, statusText, text_color = color.white, bgcolor = statusBackground, text_halign = text.align_left, text_size = size.tiny)
0946
0947 // 歷史結果固定從第 9 列開始；每組只使用 2 列，最多 3 組，剛好使用至第 14 列。
0948 if array.size(selectedEnds) > 0
0949     for rankIndex = 0 to array.size(selectedEnds) - 1
0950         int candidateEnd = array.get(selectedEnds, rankIndex)
0951         float similarity = array.get(selectedScores, rankIndex)
0952         int candidateOldestOffset = candidateEnd + patternLength - 1
0953         int distanceFromAnchor = candidateEnd - anchorOffset
0954
0955         float candidateEndClose = close[candidateEnd]
0956         float candidateFutureClose = close[candidateEnd - futureValidationBars]

```

附錄 A | 完整原始程式碼（續）

```

0957 float candidateFutureReturn = na
0958 if not na(candidateFutureClose) and not na(candidateEndClose) and math.abs(candidateEndClose) > EPSILON
0959     candidateFutureReturn := (
0960         candidateFutureClose / candidateEndClose - 1.0
0961     ) * 100.0
0962
0963 color rankColor = f_rank_color(rankIndex)
0964 color rankBackground = color.new(rankColor, 72)
0965 color rankTextColor = rankIndex == 2 ? color.white : color.black
0966 int mainRow = 9 + rankIndex * 2
0967 int detailRow = mainRow + 1
0968
0969 string rankTitle = (
0970     "歷史 #" + str.toString(rankIndex + 1) +
0971     " | " + str.toString(similarity, "#.0") + "%"
0972 )
0973 string returnText = (
0974     "後續" + str.toString(futureValidationBars) +
0975     "根" + f_signed_percent(candidateFutureReturn)
0976 )
0977
0978 table.cell(resultPanel, 0, mainRow, rankTitle, text_color = rankTextColor, bgcolor = rankBackground, text_halign = text.align_left, text_size = size.tiny)
0979 table.cell(resultPanel, 1, mainRow, returnText, text_color = rankTextColor, bgcolor = rankBackground, text_halign = text.align_left, text_size = size.tiny)
0980
0981 string startDateText = f_format_time(time[candidateOldestOffset], panelTimezone)
0982 string endDateText = f_format_time(time[candidateEnd], panelTimezone)
0983 string distanceText = "距 " + str.toString(distanceFromAnchor) + " 根"
0984
0985 table.cell(resultPanel, 0, detailRow, startDateText + " →", text_color = valueTextColor, bgcolor = normalBackground, text_halign = text.align_left, text_size =
size.tiny)
0986 table.cell(resultPanel, 1, detailRow, endDateText + " | " + distanceText, text_color = valueTextColor, bgcolor = normalBackground, text_halign = text.align_left,
text_size = size.tiny)
0987
0988
0989 // =====
0990 // 程式結尾 | © 2026 Threads(@hongshihong19) | Mozilla Public License 2.0
0991 // =====

```